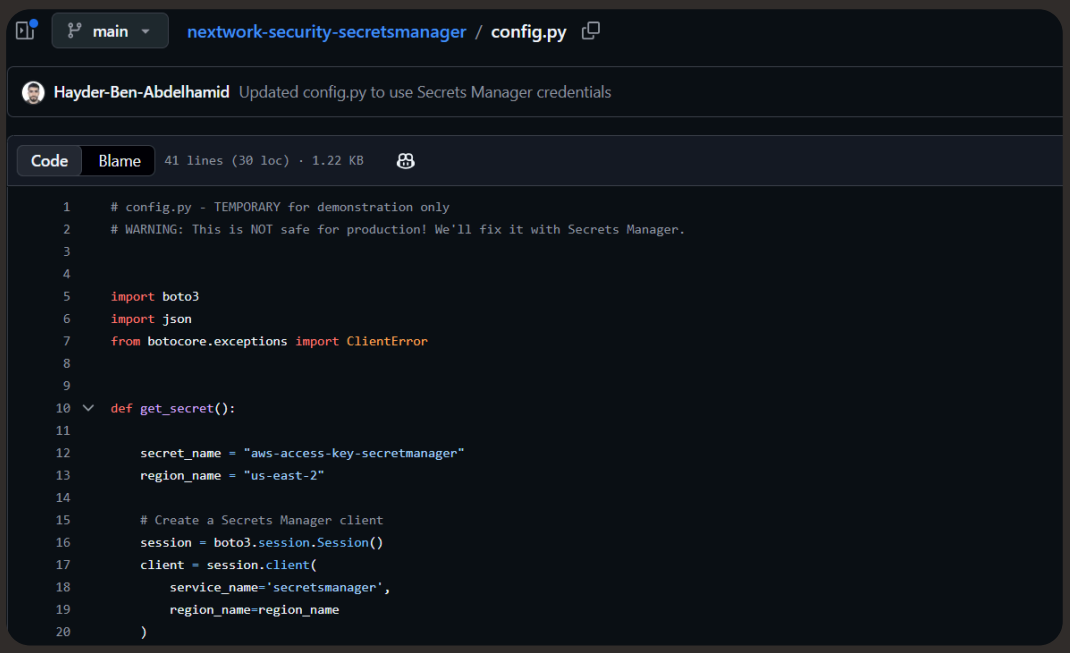
SECURE SECRETS WITH SECRETS MANAGER

By Hayder Ben Abdelhamid | October 2026



Introducing Today's Project!

In this project, I will demonstrate how to use AWS Secretes Manager for secure credentials management. I'm doing this project to learn how to protect our credentials when i connect live (in production) apps to our AWS environment and databases etc.

Tools and concepts

Services I used were Secrets Manager, GitHub, S3, IAM. Key concepts I learnt include GitHub secret scanning, the risks of hardcoding cdredentials, using git rebase to rewrite the commit history of a repository, forking a repository.

Project reflection

This project took me approximately 2H. The most challenging part was git rebase (an extra bit of the project to push the code to our repository). It was most rewarding to review the git commit history, and see that you can really find AWS credentials in the history (even if the most updated version of that file no longer expose those credentials).

I chose to do this project today because i wanted to leran more about protecting/securing the code using services like AWS Secrets Manger. This project met the goal that i'm using the best practise crdentials management. This project also helped us build the skills in Git like rebasing along the way.

Hardcoding credentials

In this project, a sample web app is exposing the developers AWS credentials i.e. Their access key ID and secret access key. It is unsafe to harcode credentials becauseonce the code is made public in repository anyone can get access to those credentials and access to our AWS environement delete resources ans steal data .

I've set up the initial configuration with a set of Access key ID and Secret Access Key. These credentials are just examples because using test credentials preverts exposing my actual AWS credentials access while doing my project.

```
# config.py - TEMPORARY for demonstration only
# WARNING: This is NOT safe for production! We'll fix it with Secrets Manager.

AWS_ACCESS_KEY_ID = "AKIAW3MEFRAFTQM5FHKE"
AWS_SECRET_ACCESS_KEY = "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"
AWS_REGION = "us-east-2"
```

Using my own AWS credentials

As an extension for this project, I also decided to run the web app locally using our AWS credentials, to set up my virtual environment, I installed packages like boto3, uvicorn and fastapi which are packages that our app's main file (app.py) uses to connect to the web app to AWS.

When I first ran the app, I ran into an InvalidAccessKeyId error because i was using test credentials inside config.py, those credentials don't actually point to a real AWS account.

To resolve the 'InvalidAccessKeyId' error, I updated the configuration file (config.py) to use my account's access key Id and secrte access key and no more test credentials.


```
{
  "buckets": [
    "cf-templates-1hdchjg9i7nh4-eu-west-3",
    "cf-templates-1hdchjg9i7nh4-us-east-2",
    "nextwork-guardduty-project-hayder-thesecurebucket-cebnzpuxhzys",
    "test-name-for-security"
  ]
}
```

Pushing Insecure Code to GitHub

Once I updated the web app code with credentials, I forked the repository code into your pubilic repo. A fork is different from a clone because it also creates a new repository in your GitHub account (where as a clone just copies that code into your local computer).

To connect my local repository to the forked repository, I run the command 'gir remote set-url origin'. Then I used git add and git commit to save the changes i made to config.py. Finally, git push uploads those chnages to the remote origin (i.e the forked repository).

GitHub blocked my push because its secrets scanning features detected that i'm hardcoding AWS creadentials. This is a good security feature because it preverts us from revealing sensetives information in a public repository

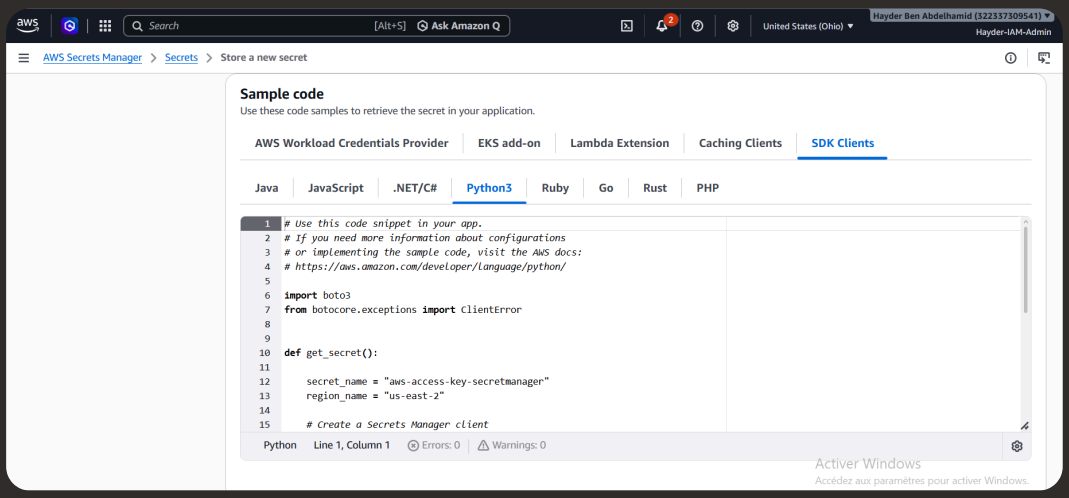
```
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 853 bytes | 106.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote: error: GH013: Repository rule violations found for refs/heads/main.
remote:
remote: - GITHUB PUSH PROTECTION
remote:   ——————————————————————————————
remote:     Resolve the following violations before pushing again
remote:
remote:     - Push cannot contain secrets
remote:
remote:
remote:      (?) Learn how to resolve a blocked push
remote:      https://docs.github.com/code-security/secret-scanning/working-with-secret-scanning-and-push-protection/working-with-push-protection-from-the-co
mmand-line#resolving-a-blocked-push
remote:
remote:
remote:       —— Amazon AWS Access Key ID ——————————————————
remote:        locations:
remote:          - commit: d3d951aaacd475c61629d4398b014ba8e94653a1
remote:            path: config.py:4
```

Secrets Manager

Secrets Manager is an AWS service that helps us with securely storing and retrieving secrets e.g. databases credentials, API Keys and anything we deem sensitive. I'm using it to store my AWS access key ID and secret access key. Other common use cases include OAuth keys and even account IDs or URLs i dont want other people to see it in the code.

Another feature in Secrets Manager is secrets rotations which means the automatic rotation of the value os secrets on a consistent schedule. Secrets Managers will generate new value for thoses secrets and It's useful in situations where you'd want to minimise how long an attacker has access your databases/API keys if it ever infiltrates your credentials/serveces.

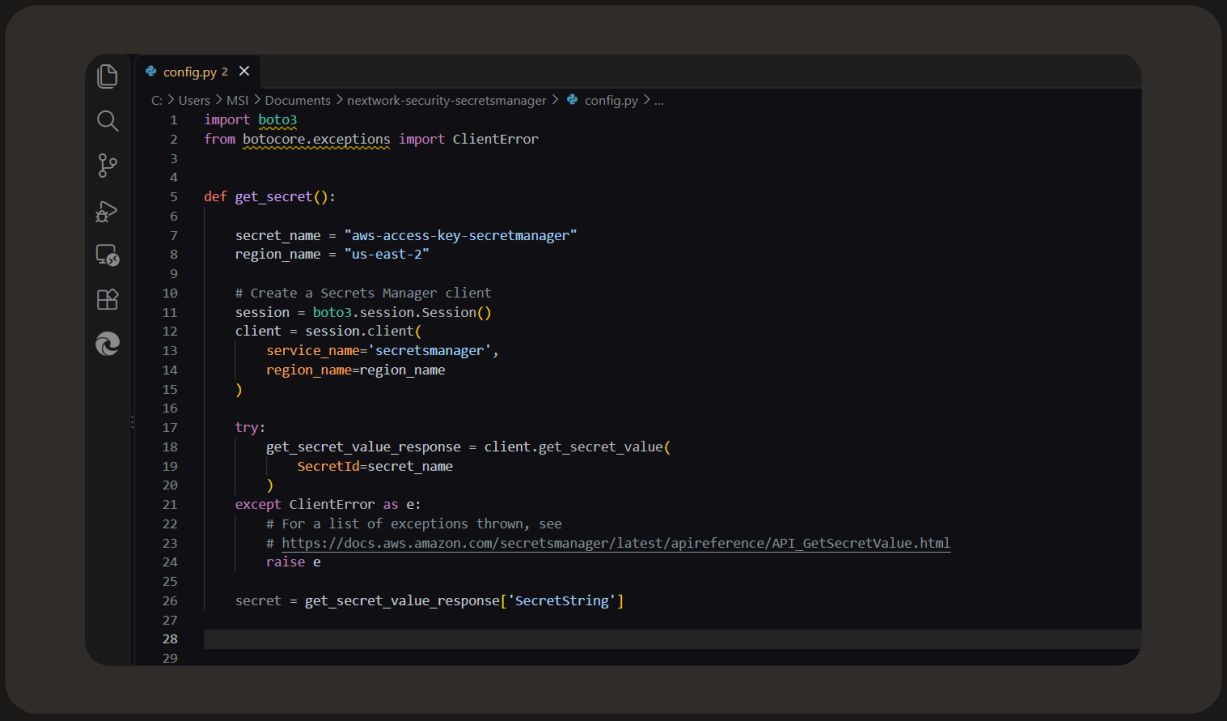
Secrets Manager provides sample code in various languages, like Java, JavaScript, Python 3 and more. This is helpful because we can use this code samples directly in our config.py file to update the way we retrieve credentials. Now i can use functions to connect to Secrets Manager.



Updating the web app code

I updated the config.py file to retrieve the secret stored in Secret Manger progromatically. The get_secret() function will set up a connection to Secret Manger (using boto3) and then will try to retrieve the value of the secret (with the secret name that i created when i used when i set it up) and store the secret's value in a variable called 'secret'

I also added code to config.py to extract the value of the secret that i stored in Secrets Manger. This is important because the configuration file now is no longer has hardcoded credentials inside. It uses the function suggested by the code sample and spilts up the secret's value to store the access key ID and secure access key in the same variable that our app.py will continue to use



Rebasing the repository

Git rebasing is the process of changing the commit history of our repository. I used it to drop the commit where i hardcoded the AWS credentials. This was necessary because the AWS credentials are still living somewhere in my repository commit history otherwise (and if it's living in the commit history, it's still discoverable to attackers).

A merge conflict occurred during rebasing because Git now confused about whether it should also remove all the changes to config.py i made in a commit AFTER the commit i wanted to drop. I resolved the merge conflict by going directly into the config.py file, and manually removing all the parts i want to remove (hardcoded credentials) and keeping the code from Secrets Manger.

Once the merge conflict was resolved, I verified thet the GitHub repository is NOT exposing any of the credentials.
I visited the config.py file in the forked repository, and can see that it uses code to extract the credentials insteaded of any hardcoding. I also successfully pushed the code without any blocking from GitHub Secrets Scanning.

